

MS SQL Joins - Part 1

By Don Schlichting

Introduction

This article explores various methods of Joining, where data from multiple tables are returned in one record set by defining logical relationships.

Microsoft SQL Server, being designed as a relational database, offers performance and integrity gains if data is organized into related tables, as opposed to placing all the data into one large table. For example, in a sales application, the customer information would be kept in a separate table from the product data. Then a third table, Sales, would be created to record information combined by the two. A benefit of this data splitting is increased performance, but the gain comes at the cost complexity. This complexity becomes apparent when extracting data out of the multiple tables for a report. This article will introduce various methods of extracting data from multiple related tables with one Select statement using Joins.

Why Use Multiple Tables

The image below shows a table where customer purchase information is recorded into one single table. Creating a sales report from this data would be very easy.

CustName	CustAddr	CustCity	Product1Name	Product1Price	Product2Name	Product2Price
Mr Smith	101 Elm St	Milwaukee	Red WigIt	\$100	Blue WigIt	\$130

```
SELECT CustName, (Product1Price + Product2Price) as SaleAmt
FROM myTable
```

Although the select query is simple, this design leads to multiple problems. First is the problem of how many products could be recorded in a single transaction. In this example, two products can be accommodated, but that is probably not enough for most orders. Therefore, how many product columns should be created, 10, 20? Eventually the max will probably be encountered in production causing errors.

The other single table possibility is to create a row for each product as shown below, which would avoid the above problem of a max purchases limit.

CustomerName	CustomerAddress	ProductPurchas...	Amount1
Mr Smith	14 Elm St	Blue WigIt	\$100
Mr Smith	14 Elm St	Green WigIt	\$130

Here, a new row is created for each item purchased. The problem is the customer information is being duplicated for each purchased item. If the database gets large, and the customers address needs changing, this update could be a performance problem. In addition, if the data is being changed from an application rather than a stored procedure, integrity could be an issue. The same issue will appear for each piece of duplicated data, Customer Name, Address, City, Zip, Product Description, Serial Number, UPC code, etc. This example only included two types of information, Customer and Product. If Sales Person,

Store Location and Payment Information also needed to be included, the design flaws will be quickly become apparent.

Splitting One table into Many

Taking the above example, the one Sales table can be split into three related tables, Customers, Products, and Sales as shown below.



See Normalization in BOL for concepts and forms of table splits. Now that there are several tables, the following examples will demonstrate joining them in SQL statements. Use the following statement to populate test data:

```
USE MASTER;
GO
CREATE DATABASE JoinTest;
GO
USE JoinTest;
GO
CREATE TABLE Customers
(CustomerName varchar(50),Addr varchar(50),City varchar(50) );
CREATE TABLE Products
(ProductName varchar(50),Descr varchar(50),Price money );
CREATE TABLE Sales
(SaleDate datetime, ProductName varchar(50),
CustomerName varchar(50), Quantity int );
GO
INSERT INTO Customers VALUES ('Mr Smith', '123 Elm St', 'Milwaukee');
INSERT INTO Customers VALUES ('Mr Jones', '456 South St', 'Chicago');
INSERT INTO Customers VALUES ('Mr Brown', '789 Oak St', 'St Louis');
INSERT INTO Products VALUES ('Green Widgeit', 'Lagre Green', 100);
INSERT INTO Products VALUES ('Blue Widgeit', 'XLagre Blue', 130);
INSERT INTO SALES VALUES ('01/01/07', 'Green Widgeit', 'Mr Smith', 1);
INSERT INTO SALES VALUES ('02/02/07', 'Green Widgeit', 'Mr Smith', 2);
```

```
INSERT INTO SALES VALUES ('02/02/07', 'Blue Widgit', 'Mr Smith', 3);
INSERT INTO SALES VALUES ('01/01/07', 'Blue Widgit', 'Mr Jones', 1);
```

INNER JOIN

There are several different types of join statements. In this first example, an INNER JOIN will be used to show all the sale dates of each customer. This type of join is used to return data from all tables specified in a statement that are true to some type of comparison.

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Customers
INNER JOIN Sales ON Customers.CustomerName = Sales.CustomerName;
```

CustomerName	SaleDate
Mr Smith	2007-01-01 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000
Mr Jones	2007-01-01 00:00:00.000

In the example, rows are returned when a Customer Name in the Customer table is equal to the same name in the Sales table. The syntax starts like a normal SELECT statement by indicating which columns should be returned. The FROM clause specifies the JOIN of an INNER type and how the two tables are related. In this case, on the Customer Name. Notice the customer Mr. Brown was not returned because there are no matching sales. With an INNER JOIN, the order each table is noted is not important. The statement could also be written with Sales before Customers as shown below.

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Sales
INNER JOIN Customers ON Customers.CustomerName = Sales.CustomerName
```

Another item worth noting is that a JOIN does not need to be done on primary or foreign key. Any column can be used. This example is joined on a non-key Varchar. Joining on a primary key however, will deliver performance gains. Joining on an Int rather than Text is also recommended. Another advantage to joining on primary keys is that you are assured there are no duplicates where you don't expect them. In the above example, multiple Sales records for each customer are expected, but only one record for Mr. Smith in the Customers table would be considered normal. If Customer Name was a Primary Key, we would be assured each was unique. This is important because the number of rows returned by an INNER JOIN on two tables in the product of Table 1 rows x Table 2 rows. So in the example, 1 Customer record x 3 sales records = 3 rows returned. However, if there was a duplicate Mr. Smith in the customer table, 6 rows would be returned, 2 Customers x 3 Sales, as pictured below.

CustomerName	SaleDate
Mr Smith	2007-01-01 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000
Mr Jones	2007-01-01 00:00:00.000
Mr Smith	2007-01-01 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000
Mr Smith	2007-02-02 00:00:00.000

An INNER JOIN can also be written into a WHERE clause as shown below.

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Sales, Customers
WHERE Customers.CustomerName = Sales.CustomerName
```

Conclusion

This first article introduced the INNER JOIN for combining two tables. Future articles will expand this concept into joining several tables at once, rather than just two, and using joins in UPDATE and DELETE statements. OUTER joins, where non-matching rows are returned will also be covered.